

KPC Revenue Assurance & Demurrage Platform

Project Documentation

Repository: github.com/doinggbits/kpc-revenue-assurance-platform **License:** MIT

1. Overview

The KPC Revenue Assurance & Demurrage Platform is a production-shaped system built for Kenya Pipeline Company (KPC) that automates the conversion of gate telemetry into verified demurrage invoices. It combines a deterministic calculation engine, a rules-based compliance matrix, role-based billing and dispute workflows, an immutable audit trail, and a simulated SAP S/4HANA posting integration — all fronted by a Next.js portal styled to KPC's brand.

Core capabilities:

- Deterministic 12-step demurrage calculation engine
- 6-rule compliance scoring matrix
- Role-based billing and dispute resolution workflows
- Immutable, append-only audit logging
- Simulated SAP S/4HANA posting (OData billing payload generation)
- CSV-based gate-log ingestion with automatic recalculation
- Anomaly detection and forecasting (ML module)
- Printable/exportable invoices (HTML/CSV)

All monetary figures are denominated in Kenyan Shillings (KES), scaled to realistic contract rates for a Kenyan logistics operation.

2. Architecture

Next.js 16 (App Router, Tailwind v4) FastAPI (Python) SQLite (WAL mode)
portal + public site <---> auth, calculation, <---> persisted volume,
real KPC design tokens compliance, billing append-only audit log

Layer	Technology
Frontend	Next.js 16 (App Router), Tailwind CSS v4
Backend	FastAPI (Python)

Layer	Technology
Database	SQLite, WAL mode, persisted in a named Docker volume (<code>kpc-db-data</code>)
Auth	JWT + bcrypt, role-based access control (RBAC)

The database survives restarts/rebuilds. Seed data is generated only the first time the database is empty; it is never wiped on subsequent restarts.

3. Project Layout

```

kpc-platform/
├── backend-service/
│   ├── backend/
│   │   ├── api/app.py      # FastAPI routes
│   │   ├── auth/          # JWT, bcrypt, RBAC dependencies
│   │   ├── calculation/   # Pure 12-step demurrage engine
│   │   ├── compliance/    # 6-rule scoring engine
│   │   ├── db/            # Schema, repository, seed data
│   │   ├── erp/           # SAP payload generator
│   │   ├── etl/           # 8-stage pipeline + CSV ingestion
│   │   ├── export/        # Printable invoice HTML
│   │   ├── ml/            # Anomaly detection, forecasting
│   │   └── audit/         # Immutable audit logger
│   ├── frontend/
│   │   ├── app/
│   │   │   ├── page.tsx    # Public landing page
│   │   │   ├── login/, signup/ # Auth pages
│   │   │   └── portal/     # Protected app (dashboard, movements,
│   │   │                       # billing, compliance, disputes, etc.)
│   └── docker-compose.yml

```

4. Getting Started

4.1 Prerequisites

- **Docker + Docker Compose** (recommended), **or**
- **Manual setup:** Python 3.12+ and Node.js 20+

4.2 Run with Docker Compose (recommended)

`docker compose up`

The Dockerfiles and compose file were written and reviewed against both services' real dependencies but have not been build-tested (no Docker daemon was available at build time). Both services have been fully run and verified directly (Python venv + `npm run dev/build`). If the Docker build surfaces issues, they are most likely base-image or lockfile version mismatches rather than application bugs.

5. Demo Accounts

The database seeds six accounts on first startup, with real bcrypt-hashed passwords validated through the normal login flow. (The "quick login" buttons on the sign-in page are a convenience shortcut, not an auth bypass.)

Role	Username	Password
Ops Controller	<code>juma.mwangi</code>	<code>OpsControl@2026</code>
Billing Approver	<code>sarah.ochieng</code>	<code>BillingApprove@2026</code>
Finance Lead	<code>david.kiprop</code>	<code>FinanceLead@2026</code>
Senior Auditor	<code>grace.wambui</code>	<code>AuditTrail@2026</code>
System Admin	<code>admin.controller</code>	<code>SysAdmin@2026</code>
OMC Representative (Vivo Energy)	<code>vivo.rep</code>	<code>OmcPortal@2026</code>

New OMC representatives can self-register at `/signup`. Internal KPC roles (Ops, Billing, Finance, Auditor, Admin) are provisioned only by an Admin — self-signup for these roles is disabled and enforced server-side.

 **Security note:** These are demo credentials seeded for evaluation purposes. They should be rotated or removed before any non-demo deployment, and this table should not be published outside the project team.

6. What's Real vs. Simulated

Real / fully functional:

- Authentication (JWT + bcrypt)
- Calculation and compliance engines
- All database reads/writes
- Audit trail
- CSV ingestion (upserts + recalculation)
- RBAC enforcement
- CSV/HTML exports

Simulated by design:

- The SAP S/4HANA push (`POST /api/v1/invoices/{id}/sync-erp`) generates a correctly-shaped OData billing payload and a reference number, but does not call a real SAP instance (none exists in the demo environment). The response structure matches what a genuine integration would exchange, so it can serve as a contract reference for a future real integration.
-

7. Data Ingestion — Gate-Log CSV Upload

The Reports & Exports page accepts gate-log CSVs with the following header:

trip_id,truck_plate,carrier,customer_id,contract_id,depot_id,product,scheduled_gate_in,gate_in,gate_out,status

Rules:

- `customer_id`, `contract_id`, and `depot_id` must match existing records (validate against `GET /api/v1/contracts` and `GET /api/v1/depots`).
 - Re-uploading the same `trip_id` updates the existing movement rather than creating a duplicate.
 - Any row with a populated `gate_out` value triggers a full calculation and compliance check immediately upon ingestion.
-

8. Core Engines

8.1 Calculation Engine ([backend/calculation/](#))

A pure, deterministic 12-step engine that computes demurrage charges from gate movement data (scheduled vs. actual gate-in/gate-out times, contract terms, etc.).

8.2 Compliance Engine ([backend/compliance/](#))

A 6-rule scoring matrix that evaluates each movement/invoice against compliance criteria.

8.3 ETL Pipeline ([backend/etl/](#))

An 8-stage pipeline handling CSV ingestion, validation, upserts, and triggering recalculation.

8.4 ML Module ([backend/ml/](#))

Anomaly detection and forecasting over movement and billing data.

8.5 ERP Integration ([backend/erp/](#))

Generates SAP S/4HANA-compatible OData billing payloads (simulated push — see Section 6).

8.6 Audit Module ([backend/audit/](#))

Immutable, append-only logging of system actions for traceability and compliance review.

9. Roles & Workflows

The system implements role-based billing and dispute workflows across the following roles:

- **Ops Controller** — manages gate movements and operational data
 - **Billing Approver** — reviews and approves billing/invoices
 - **Finance Lead** — oversees financial reporting and postings
 - **Senior Auditor** — reviews the immutable audit trail
 - **System Admin** — provisions internal KPC accounts and manages system configuration
 - **OMC Representative** — external customer-facing role (e.g., Vivo Energy), can self-register via [/signup](#)
-

10. Currency

All monetary figures throughout the system are denominated in Kenyan Shillings (KES), scaled to realistic contract rates for a Kenyan logistics operation.

11. Open Items / Notes for the Team

- Docker build has not yet been build-tested in a live Docker daemon environment — verify on first real deployment.
- SAP integration is currently simulated; a real SAP S/4HANA endpoint integration is a likely next step before production use.
- Demo credentials (Section 5) should be rotated/removed prior to any external or production deployment.

This document was compiled from the project README and repository structure. For implementation-level detail (exact compliance rule definitions, calculation formulas, API schemas), refer to the source in [backend-service/backend/calculation/](#), [backend-service/backend/compliance/](#), and the live API docs at [/docs](#) once the backend is running.